



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Технології розподілених обчислень»
Комп'ютерний практикум №8
«Розробка алгоритмів для розподілених систем клієнт-серверної
архітектури»

Виконав:

Студент III курсу
гр. ІІІ-33
Соколов О. В.

Прийняв:

Дифучин А. Ю.
“18” квітня 2026 р.

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №7

Тема: Розробка алгоритмів для розподілених систем клієнт-серверної архітектури

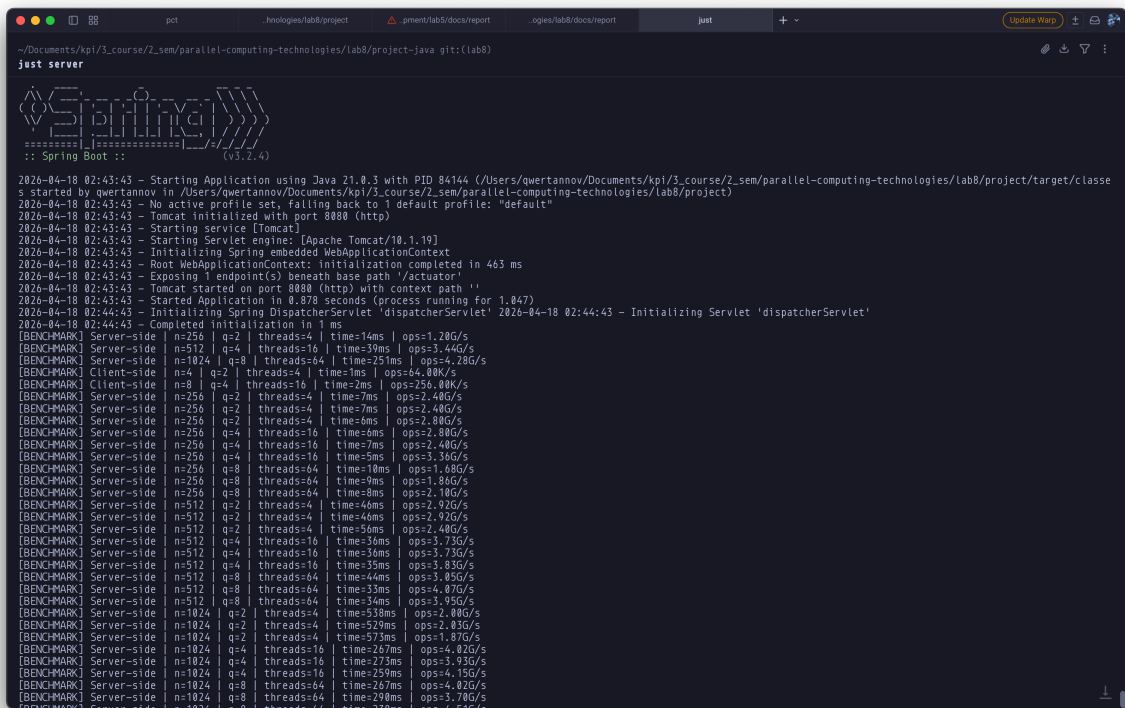
Завдання:

1. Розробити веб-застосування клієнт-серверної архітектури, що реалізує алгоритм множення матриць (або інший обчислювальний алгоритм, який був Вами реалізований іншими методами розподілених обчислень в рамках курсу «Паралельні та розподілені обчислення») на стороні сервера з використанням паралельних обчислень. Розгляньте два варіанти реалізації 1) дані для обчислень знаходяться на сервері та 2) дані для обчислень знаходяться на клієнтській частині застосування. 60 балів.
2. Дослідити швидкість виконання запиту користувача при різних обсягах даних. 20 балів.
3. Порівняти реалізацію алгоритму в клієнт-серверній системи та в розподіленій системі з рівноправними процесорами. 20 балів.

ВИКОНАННЯ

Завдання 1

Реалізовано серверну частину з паралельним множенням за блочним алгоритмом Фокса на сітці $q \times q$ (умова сумісності: n ділиться на q). Клієнт взаємодіє із сервером через HTTP з тілом запиту у форматі JSON. Передбачено перевірку доступності сервісу.



```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab8/project-java git:(lab8)
just server

:: Spring Boot ::
(v3.2.4)

2026-04-18 02:43:43 - Starting Application using Java 21.0.3 with PID 84144 (/Users/quertannov/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab8/project/target/classes
s started by quertannov in /Users/quertannov/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab8/project)
2026-04-18 02:43:43 - No active profile set, falling back to 1 default profile: "default"
2026-04-18 02:43:43 - Tomcat initialized with port 8080 (http)
2026-04-18 02:43:43 - Starting service [Tomcat]
2026-04-18 02:43:43 - Starting Servlet engine: [Apache Tomcat/10.1.19]
2026-04-18 02:43:43 - Initializing Spring embedded WebApplicationContext
2026-04-18 02:43:43 - Root WebApplicationContext: initialization completed in 463 ms
2026-04-18 02:43:43 - Exposing 1 endpoint(s) beneath base path '/actuator'
2026-04-18 02:43:43 - Tomcat started on port 8080 (http) with context path ''
2026-04-18 02:43:43 - Started Application in 0.878 seconds (process running for 1.047)
2026-04-18 02:44:43 - Initializing Spring DispatcherServlet 'dispatcherServlet' 2026-04-18 02:44:43 - Initializing Servlet 'dispatcherServlet'
2026-04-18 02:44:43 - Completed initialization in 1 ms
[BENCHMARK] Server-side | n=256 | q=2 | threads=4 | time=14ms | ops=1.20G/s
[BENCHMARK] Server-side | n=512 | q=4 | threads=16 | time=39ms | ops=3.44G/s
[BENCHMARK] Server-side | n=1024 | q=8 | threads=64 | time=25ms | ops=4.28G/s
[BENCHMARK] Client-side | n=4 | q=2 | threads=4 | time=1ms | ops=64.00K/s
[BENCHMARK] Client-side | n=8 | q=4 | threads=16 | time=2ms | ops=256.00K/s
[BENCHMARK] Server-side | n=256 | q=2 | threads=4 | time=7ms | ops=2.40G/s
[BENCHMARK] Server-side | n=256 | q=2 | threads=4 | time=6ms | ops=2.80G/s
[BENCHMARK] Server-side | n=256 | q=4 | threads=16 | time=6ms | ops=2.80G/s
[BENCHMARK] Server-side | n=256 | q=4 | threads=16 | time=7ms | ops=2.40G/s
[BENCHMARK] Server-side | n=256 | q=4 | threads=16 | time=5ms | ops=3.36G/s
[BENCHMARK] Server-side | n=256 | q=8 | threads=64 | time=10ms | ops=1.68G/s
[BENCHMARK] Server-side | n=256 | q=8 | threads=64 | time=9ms | ops=1.86G/s
[BENCHMARK] Server-side | n=256 | q=8 | threads=64 | time=8ms | ops=2.10G/s
[BENCHMARK] Server-side | n=512 | q=2 | threads=4 | time=46ms | ops=2.92G/s
[BENCHMARK] Server-side | n=512 | q=2 | threads=4 | time=56ms | ops=2.40G/s
[BENCHMARK] Server-side | n=512 | q=4 | threads=16 | time=36ms | ops=3.73G/s
[BENCHMARK] Server-side | n=512 | q=4 | threads=16 | time=36ms | ops=3.73G/s
[BENCHMARK] Server-side | n=512 | q=4 | threads=16 | time=35ms | ops=3.83G/s
[BENCHMARK] Server-side | n=512 | q=8 | threads=64 | time=44ms | ops=3.05G/s
[BENCHMARK] Server-side | n=512 | q=8 | threads=64 | time=33ms | ops=4.07G/s
[BENCHMARK] Server-side | n=512 | q=8 | threads=64 | time=34ms | ops=3.95G/s
[BENCHMARK] Server-side | n=1024 | q=2 | threads=4 | time=538ms | ops=2.08G/s
[BENCHMARK] Server-side | n=1024 | q=2 | threads=4 | time=529ms | ops=2.03G/s
[BENCHMARK] Server-side | n=1024 | q=4 | threads=16 | time=579ms | ops=1.87G/s
[BENCHMARK] Server-side | n=1024 | q=4 | threads=16 | time=267ms | ops=4.02G/s
[BENCHMARK] Server-side | n=1024 | q=4 | threads=16 | time=273ms | ops=3.93G/s
[BENCHMARK] Server-side | n=1024 | q=4 | threads=16 | time=259ms | ops=4.15G/s
[BENCHMARK] Server-side | n=1024 | q=8 | threads=64 | time=267ms | ops=4.02G/s
[BENCHMARK] Server-side | n=1024 | q=8 | threads=64 | time=248ms | ops=3.78G/s
[BENCHMARK] Server-side | n=1024 | q=8 | threads=64 | time=276ms | ops=3.54G/s
```

Завдання 2

```
~/Documents/ps1/2_course/2_sem/parallel-computing-technologies/lab8/project git:(lab8) (10.303s)
just benchmark
Iteration 1: computation=273ms, total=462ms
Iteration 2: computation=259ms, total=446ms
Iteration 3: computation=259ms, total=446ms
Average: 260ms, Min: 259ms, Max: 273ms

Running benchmark: n=1024, q=8 (threads=64)
Iteration 1: computation=267ms, total=459ms
Iteration 2: computation=240ms, total=427ms
Iteration 3: computation=238ms, total=423ms
Average: 265ms, Min: 238ms, Max: 290ms

=====
BENCHMARK SUMMARY
=====
Size (n)  Grid (q)  Threads  Avg Time (ms)  Min (ms)  Max (ms)  Throughput
-----
256      2         4         6              6          7         2.80G ops/s
256      4         16        6              5          7         2.80G ops/s
256      8         64         9              8         10         1.86G ops/s
512      2         4         49             46         56         2.74G ops/s
512      4         16        35             35         44         3.83G ops/s
512      8         64        37             33         44         3.63G ops/s
1024     2         4         546            529        573         1.97G ops/s
1024     4         16        266            259        273         4.84G ops/s
1024     8         64        265            238        290         4.05G ops/s
=====

PERFORMANCE ANALYSIS
=====
Matrix size n=256:
q=2, threads=4: 1.00x speedup vs q=2, 2.80G ops/s
q=4, threads=16: 1.00x speedup vs q=2, 2.80G ops/s
q=8, threads=64: 0.67x speedup vs q=2, 1.86G ops/s

Matrix size n=512:
q=2, threads=4: 1.00x speedup vs q=2, 2.74G ops/s
q=4, threads=16: 1.48x speedup vs q=2, 3.83G ops/s
q=8, threads=64: 1.32x speedup vs q=2, 3.63G ops/s

Matrix size n=1024:
q=2, threads=4: 1.00x speedup vs q=2, 1.97G ops/s
q=4, threads=16: 2.05x speedup vs q=2, 4.84G ops/s
q=8, threads=64: 2.06x speedup vs q=2, 4.85G ops/s

Analyze the benchmark results and explain the performance scaling trends observed for different matrix sizes and thread counts.
LeI-computing-technologies/lab8/project lab8 21 • +1551
Run commands
```

Розглядалися розміри $n \in \{256, 512, 1024\}$ і параметри сітки $q \in \{2, 4, 8\}$ (усі пари з діленням n на q). Для кожної пари (n, q) виконано три послідовні запити; у таблиці наведено середній, мінімальний і максимальний час саме обчислення на сервері ($t_{\text{обч}}$, мс), а також повний час HTTP-запиту ($t_{\text{зап}}$, мс) за першими трьома ітераціями серії (для ілюстрації накладних витрат транспорту та обробки запиту порівняно з $t_{\text{обч}}$).

Таблиця 2.1 – Час обчислення $t_{\text{обч}}$ (середнє / мін / макс за трьома прогонами).

n	q	Кількість потоків q^2	Середнє, мс	Мін, мс	Макс, мс
256	2	4	6	6	7
256	4	16	6	5	7
256	8	64	9	8	10
512	2	4	49	46	56

n	q	Кількість потоків q^2	Середнє, мс	Мін, мс	Макс, мс
512	4	16	35	35	36
512	8	64	37	33	44
1024	2	4	546	529	573
1024	4	16	266	259	273
1024	8	64	265	238	290

Таблиця 2.2 – Приклад: повний час запиту $t_{\text{зап}}$ проти $t_{\text{обч}}$ (мс), перший прогон у кожній конфігурації.

n	q	$t_{\text{обч}}$	$t_{\text{зап}}$
256	2	7	74
256	4	6	23
256	8	10	25
512	2	46	104
512	4	36	87
512	8	44	92
1024	2	538	736
1024	4	267	453
1024	8	267	459

Аналіз результатів.

1. При збільшенні n час обчислення зростає у кубічній залежності від лінійного розміру, що відповідає складності множення матриць.

2. Для $n = 256$ збільшення числа потоків не дає виграшу: при $q = 4$ час такий самий, як при $q = 2$, а при $q = 8$ час зростає (умовне прискорення відносно $q = 2 - 0,67\times$), тобто для малої задачі домінують накладні витрати паралелізму.

3. Для $n = 512$ перехід від $q = 2$ до $q = 4$ дає прискорення $\approx 1,40\times$, до $q = 8 - \approx 1,32\times$; подальше збільшення q дає незначний ефект порівняно з $q = 4$.

4. Для $n = 1024$ перехід від $q = 2$ до $q = 4$ і $q = 8$ дає прискорення $\approx 2,05\times$ і $\approx 2,06\times$; варіанти $q = 4$ і $q = 8$ майже рівні за середнім часом, отже, на обраній конфігурації вичерпано вигоду від додаткового паралелізму або проявляються обмеження пам'яті та синхронізації.

5. $t_{\text{зап}}$ суттєво перевищує $t_{\text{обч}}$ на першому прогоні (особливо для малих n), що показує внесок мережевого шару та обробки запиту; для

порівняння «чистої» продуктивності алгоритму слід орієнтуватися на $t_{\text{обч}}$,
а для оцінки досвіду користувача – на $t_{\text{зап}}$.

Завдання 3

Таблиця 3.1 – Узагальнене порівняння моделей.

Аспект	Клієнт-сервер	MPI, рівноправні процеси
Обчислювальна модель	багатопотоковість у межах одного сервера	багатопроцесність, окремі адресні простори
Пам'ять	спільна для потоків	розподілена; потрібні пересилання
Зовнішній інтерфейс	веб-запити, текстовий обмін даними	немає веб-шару; обмін усередині MPI
Типові накладні витрати	формування та розбір повідомлень запиту, мережа	колективні та точкові операції MPI
Масштабування	зростання потужності одного вузла	зростання кількості процесів і вузлів

Висновок

Реалізовано клієнт-серверне веб-застосування з двома сценаріями постачання даних і паралельним множенням матриць на сервері за алгоритмом Фокса. Проведено серію вимірювань для різних n і q ; показано зростання часу з n , вигоду від $q > 2$ для великих матриць та невигоду для малих.

Код програми

Посилання

на

GitHub

репозиторій:

<https://github.com/sokolovgit/parallel-computing-technologies>

```
=== ./main/java/ua/kpi/lab8/Application.java ===
package ua.kpi.lab8;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {

System.out.println("=====");
        System.out.println("Lab8: Matrix Multiplication Server (Fox
Algorithm)");

System.out.println("=====");
        SpringApplication.run(Application.class, args);
    }
}

=== ./main/java/ua/kpi/lab8/algorithm/ExecutorUtils.java ===
package ua.kpi.lab8.algorithm;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.TimeUnit;

public final class ExecutorUtils {

    private ExecutorUtils() {}

    public static void shutdownQuietly(ExecutorService executor) {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}

=== ./main/java/ua/kpi/lab8/algorithm/FoxParallelMultiplier.java ===
package ua.kpi.lab8.algorithm;

import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public final class FoxParallelMultiplier implements MatrixMultiplier {
```

```

private final int q;

public FoxParallelMultiplier(int q) {
    this.q = q;
}

@Override
public void multiply(double[][] a, double[][] b, double[][] c) throws
InterruptedException {
    int n = a.length;

    if (n == 0) {
        return;
    }

    if (q < 1 || n % q != 0) {
        throw new IllegalArgumentException("n must be divisible by
q");
    }

    int m = n / q;
    int workers = q * q;

    ExecutorService executor = Executors.newFixedThreadPool(workers);
    try {
        for (int k = 0; k < q; k++) {
            CountdownLatch latch = new CountdownLatch(workers);
            final int kf = k;
            for (int bi = 0; bi < q; bi++) {
                for (int bj = 0; bj < q; bj++) {
                    final int biF = bi;
                    final int bjF = bj;
                    executor.execute(() -> {
                        try {
                            int aRow0 = biF * m;
                            int aCol0 = ((biF + kf) % q) * m;
                            int bRow0 = ((biF + kf) % q) * m;
                            int bCol0 = bjF * m;
                            int cRow0 = biF * m;
                            int cCol0 = bjF * m;
                            blockMultiplyAdd(a, aRow0, aCol0, b,
bRow0, bCol0, c, cRow0, cCol0, m);
                        } finally {
                            latch.countDown();
                        }
                    });
                }
            }
            latch.await();
        } finally {
            ExecutorUtils.shutdownQuietly(executor);
        }
    }

    private static void blockMultiplyAdd(
        double[][] a,
        int aRow0,

```

```

        int aCol0,
        double[][] b,
        int bRow0,
        int bCol0,
        double[][] c,
        int cRow0,
        int cCol0,
        int m) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            double sum = 0.0;
            for (int kk = 0; kk < m; kk++) {
                sum += a[aRow0 + i][aCol0 + kk] * b[bRow0 + kk][bCol0
+ j];
            }
            c[cRow0 + i][cCol0 + j] += sum;
        }
    }
}

public int getGridSize() {
    return q;
}

public int getWorkerCount() {
    return q * q;
}
}

```

```

=== ./main/java/ua/kpi/lab8/algorithm/MatrixMultiplier.java ===
package ua.kpi.lab8.algorithm;

```

```

@FunctionalInterface
public interface MatrixMultiplier {

    void multiply(double[][] a, double[][] b, double[][] c) throws
InterruptedException;
}

```

```

=== ./main/java/ua/kpi/lab8/benchmark/BenchmarkRunner.java ===
package ua.kpi.lab8.benchmark;

```

```

import ua.kpi.lab8.algorithm.FoxParallelMultiplier;

```

```

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

```

```

public class BenchmarkRunner {

    private static final int[] MATRIX_SIZES = {256, 512, 1024};
    private static final int[] GRID_SIZES = {2, 4, 8};
    private static final int ITERATIONS = 3;

    private final Random random = new Random(42);

    public static void main(String[] args) throws InterruptedException {
        BenchmarkRunner runner = new BenchmarkRunner();
    }
}

```

```

        runner.runAllBenchmarks();
    }

    public void runAllBenchmarks() throws InterruptedException {
        System.out.println("\n=====");
        System.out.println("Lab8 Benchmark: Fox Parallel Matrix
        Multiplication");
        System.out.println("=====\\n");

        List<BenchmarkResult> results = new ArrayList<>();

        for (int n : MATRIX_SIZES) {
            for (int q : GRID_SIZES) {
                if (n % q == 0) {
                    BenchmarkResult result = runBenchmark(n, q);
                    results.add(result);
                }
            }
        }

        printSummary(results);
    }

    private BenchmarkResult runBenchmark(int n, int q) throws
    InterruptedException {
        System.out.printf("Running benchmark: n=%d, q=%d (threads=%d)%n",
        n, q, q * q);

        List<Long> times = new ArrayList<>();

        for (int i = 0; i < ITERATIONS; i++) {
            double[][] matrixA = generateRandomMatrix(n);
            double[][] matrixB = generateRandomMatrix(n);
            double[][] result = new double[n][n];

            FoxParallelMultiplier multiplier = new
            FoxParallelMultiplier(q);

            long startTime = System.currentTimeMillis();
            multiplier.multiply(matrixA, matrixB, result);
            long endTime = System.currentTimeMillis();

            long time = endTime - startTime;
            times.add(time);
            System.out.printf("  Iteration %d: %dms%n", i + 1, time);
        }

        long avgTime = (long)
        times.stream().mapToLong(Long::longValue).average().orElse(0);
        long minTime =
        times.stream().mapToLong(Long::longValue).min().orElse(0);
        long maxTime =
        times.stream().mapToLong(Long::longValue).max().orElse(0);

        System.out.printf("  Average: %dms, Min: %dms, Max: %dms%n%n",
        avgTime, minTime, maxTime);
    }

```

```

        return new BenchmarkResult(n, q, q * q, avgTime, minTime,
maxTime);
    }

    private double[][] generateRandomMatrix(int n) {
        double[][] matrix = new double[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = random.nextDouble() * 10;
            }
        }
        return matrix;
    }

    private void printSummary(List<BenchmarkResult> results) {
System.out.println("=====");
        System.out.println("BENCHMARK SUMMARY");

System.out.println("=====");
        System.out.printf("%-10s %-10s %-10s %-15s %-10s %-10s\n",
            "Size (n)", "Grid (q)", "Threads", "Avg Time (ms)", "Min
(ms)", "Max (ms)");
        System.out.println("-----
---");

        for (BenchmarkResult result : results) {
            System.out.printf("%-10d %-10d %-10d %-15d %-10d %-10d\n",
                result.matrixSize, result.gridSize, result.threads,
                result.avgTime, result.minTime, result.maxTime);
        }

System.out.println("=====\\n");

        printPerformanceAnalysis(results);
    }

    private void printPerformanceAnalysis(List<BenchmarkResult> results) {
        System.out.println("PERFORMANCE ANALYSIS");

System.out.println("=====");

        for (int n : MATRIX_SIZES) {
            System.out.printf("\\nMatrix size n=%d:%n", n);
            List<BenchmarkResult> sizeResults = results.stream()
                .filter(r -> r.matrixSize == n)
                .toList();

            if (!sizeResults.isEmpty()) {
                BenchmarkResult baseline = sizeResults.get(0);
                for (BenchmarkResult result : sizeResults) {
                    double speedup = (double) baseline.avgTime /
result.avgTime;

                    long operations = (long) n * n * n;
                    double opsPerSecond = operations / (result.avgTime /
1000.0);

```

```

        System.out.printf("  q=%d, threads=%d: %.2fx speedup,
%.2fM ops/s%n",
        result.gridSize, result.threads, speedup,
opsPerSecond / 1_000_000);
    }
}
}

```

```

System.out.println("\n=====");
}

```

```

    private static class BenchmarkResult {
        final int matrixSize;
        final int gridSize;
        final int threads;
        final long avgTime;
        final long minTime;
        final long maxTime;

        BenchmarkResult(int matrixSize, int gridSize, int threads, long
avgTime, long minTime, long maxTime) {
            this.matrixSize = matrixSize;
            this.gridSize = gridSize;
            this.threads = threads;
            this.avgTime = avgTime;
            this.minTime = minTime;
            this.maxTime = maxTime;
        }
    }
}

```

```

=== ./main/java/ua/kpi/lab8/benchmark/DemoRunner.java ===
package ua.kpi.lab8.benchmark;

```

```

import com.fasterxml.jackson.databind.ObjectMapper;

```

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Map;

```

```

public class DemoRunner {

```

```

    private static final String BASE_URL =
"http://localhost:8080/api/multiply";
    private static final HttpClient client = HttpClient.newHttpClient();
    private static final ObjectMapper mapper = new ObjectMapper();

```

```

    public static void main(String[] args) throws Exception {

```

```

System.out.println("=====");
        System.out.println("Lab8 Demo: Testing Matrix Multiplication
Endpoints");

```

```

System.out.println("=====");
        System.out.println();
    }
}

```

```

        healthCheck();
        serverSideDemo();
        clientSideDemo();

System.out.println("=====");
        System.out.println("Demo complete! Check server console for
benchmarks.");

System.out.println("=====");
    }

    private static void healthCheck() throws Exception {
        System.out.println("1. Health check...");
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/health"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
        System.out.println("    " + response.body());
        System.out.println();
    }

    private static void serverSideDemo() throws Exception {
        System.out.println("2. Server-side multiplication (n=256,
q=2)...");
        sendServerRequest(256, 2);
        System.out.println();

        System.out.println("3. Server-side multiplication (n=512,
q=4)...");
        sendServerRequest(512, 4);
        System.out.println();

        System.out.println("4. Server-side multiplication (n=1024,
q=8)...");
        sendServerRequest(1024, 8);
        System.out.println();
    }

    private static void clientSideDemo() throws Exception {
        System.out.println("5. Client-side multiplication (4x4 matrices,
q=2)...");
        double[][] matrixA = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12},
            {13, 14, 15, 16}
        };
        double[][] matrixB = {
            {1, 0, 0, 0},
            {0, 1, 0, 0},
            {0, 0, 1, 0},
            {0, 0, 0, 1}
        };
    }

```



```

        sendClientRequest(matrixA, matrixB, 2);
        System.out.println();

        System.out.println("6. Client-side multiplication (8x8 matrices,
q=4)...");
        double[][] matrixA8 = new double[8][8];
        double[][] matrixB8 = new double[8][8];
        for (int i = 0; i < 8; i++) {
            for (int j = 0; j < 8; j++) {
                matrixA8[i][j] = i + j + 1;
                matrixB8[i][j] = (i == j) ? 1 : 0;
            }
        }
        sendClientRequest(matrixA8, matrixB8, 4);
        System.out.println();
    }

    private static void sendServerRequest(int n, int q) throws Exception {
        Map<String, Integer> requestBody = Map.of("n", n, "q", q);
        String json = mapper.writeValueAsString(requestBody);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/server"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
        Map<String, Object> result = mapper.readValue(response.body(),
Map.class);

        System.out.printf("    Time: %sms, Size: %s, Grid: %s, Threads: %s,
Source: %s%n",
            result.get("computationTimeMs"),
            result.get("matrixSize"),
            result.get("gridSize"),
            result.get("threadsUsed"),
            result.get("dataSource"));
    }

    private static void sendClientRequest(double[][] matrixA, double[][][]
matrixB, int q) throws Exception {
        Map<String, Object> requestBody = Map.of(
            "matrixA", matrixA,
            "matrixB", matrixB,
            "q", q
        );
        String json = mapper.writeValueAsString(requestBody);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/client"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

```

```

        Map<String, Object> result = mapper.readValue(response.body(),
Map.class);

        System.out.printf("    Time: %sms, Size: %s, Grid: %s, Threads: %s,
Source: %s\n",
            result.get("computationTimeMs"),
            result.get("matrixSize"),
            result.get("gridSize"),
            result.get("threadsUsed"),
            result.get("dataSource"));
    }
}

=== ./main/java/ua/kpi/lab8/benchmark/HttpBenchmarkRunner.java ===
package ua.kpi.lab8.benchmark;

import com.fasterxml.jackson.databind.ObjectMapper;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class HttpBenchmarkRunner {

    private static final String BASE_URL =
"http://localhost:8080/api/multiply/server";
    private static final HttpClient client = HttpClient.newHttpClient();
    private static final ObjectMapper mapper = new ObjectMapper();

    private static final int[] MATRIX_SIZES = {256, 512, 1024};
    private static final int[] GRID_SIZES = {2, 4, 8};
    private static final int ITERATIONS = 3;

    public static void main(String[] args) throws Exception {

System.out.println("\n=====
");
        System.out.println("Lab8 HTTP Benchmark Suite");

System.out.println("=====
");

        List<BenchmarkResult> results = new ArrayList<>();

        for (int n : MATRIX_SIZES) {
            for (int q : GRID_SIZES) {
                if (n % q == 0) {
                    BenchmarkResult result = runBenchmark(n, q);
                    results.add(result);
                }
            }
        }

        printSummary(results);

```

```

    }

    private static BenchmarkResult runBenchmark(int n, int q) throws
Exception {
        System.out.printf("Running benchmark: n=%d, q=%d (threads=%d)%n",
n, q, q * q);

        List<Long> times = new ArrayList<>();

        for (int i = 0; i < ITERATIONS; i++) {
            Map<String, Integer> requestBody = Map.of("n", n, "q", q);
            String json = mapper.writeValueAsString(requestBody);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(BASE_URL))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

            long startTime = System.currentTimeMillis();
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
            long totalTime = System.currentTimeMillis() - startTime;

            Map<String, Object> result = mapper.readValue(response.body(),
Map.class);
            long computationTime = ((Number)
result.get("computationTimeMs")).longValue();

            times.add(computationTime);
            System.out.printf("  Iteration %d: computation=%dms,
total=%dms%n",
                i + 1, computationTime, totalTime);
        }

        long avgTime = (long)
times.stream().mapToLong(Long::longValue).average().orElse(0);
        long minTime =
times.stream().mapToLong(Long::longValue).min().orElse(0);
        long maxTime =
times.stream().mapToLong(Long::longValue).max().orElse(0);

        System.out.printf("  Average: %dms, Min: %dms, Max: %dms%n%n",
avgTime, minTime, maxTime);

        return new BenchmarkResult(n, q, q * q, avgTime, minTime,
maxTime);
    }

    private static void printSummary(List<BenchmarkResult> results) {
        System.out.println("=====");
        System.out.println("BENCHMARK SUMMARY");

        System.out.println("=====");
        System.out.printf("%-10s %-10s %-10s %-15s %-10s %-10s %-15s%n",
            "Size (n)", "Grid (q)", "Threads", "Avg Time (ms)", "Min
(ms)", "Max (ms)", "Throughput");
    }

```

```

        System.out.println("-----");
        -----");

        for (BenchmarkResult result : results) {
            long operations = (long) result.matrixSize * result.matrixSize
* result.matrixSize;
            double opsPerSecond = operations / (result.avgTime / 1000.0);
            String throughput = formatOps(opsPerSecond);

            System.out.printf("%-10d %-10d %-10d %-15d %-10d %-10d %-
15s%n",
                                result.matrixSize, result.gridSize, result.threads,
                                result.avgTime, result.minTime, result.maxTime,
throughput);
        }

System.out.println("=====\\n"
);

        printPerformanceAnalysis(results);
    }

    private static void printPerformanceAnalysis(List<BenchmarkResult>
results) {
        System.out.println("PERFORMANCE ANALYSIS");

System.out.println("=====");

        for (int n : MATRIX_SIZES) {
            System.out.printf("\\nMatrix size n=%d:%n", n);
            List<BenchmarkResult> sizeResults = results.stream()
                .filter(r -> r.matrixSize == n)
                .toList();

            if (!sizeResults.isEmpty()) {
                BenchmarkResult baseline = sizeResults.get(0);
                for (BenchmarkResult result : sizeResults) {
                    double speedup = (double) baseline.avgTime /
result.avgTime;
                    long operations = (long) n * n * n;
                    double opsPerSecond = operations / (result.avgTime /
1000.0);
                    System.out.printf("  q=%d, threads=%d: %.2fx speedup
vs q=%d, %s%n",
                                    result.gridSize, result.threads, speedup,
baseline.gridSize,
                                    formatOps(opsPerSecond));
                }
            }
        }

System.out.println("\\n=====");
    }

    private static String formatOps(double opsPerSecond) {

```

```

        if (opsPerSecond >= 1_000_000_000) {
            return String.format("%.2fG ops/s", opsPerSecond /
1_000_000_000);
        } else if (opsPerSecond >= 1_000_000) {
            return String.format("%.2fM ops/s", opsPerSecond / 1_000_000);
        } else if (opsPerSecond >= 1_000) {
            return String.format("%.2fK ops/s", opsPerSecond / 1_000);
        } else {
            return String.format("%.2f ops/s", opsPerSecond);
        }
    }
}

```

```

private static class BenchmarkResult {
    final int matrixSize;
    final int gridSize;
    final int threads;
    final long avgTime;
    final long minTime;
    final long maxTime;

    BenchmarkResult(int matrixSize, int gridSize, int threads, long
avgTime, long minTime, long maxTime) {
        this.matrixSize = matrixSize;
        this.gridSize = gridSize;
        this.threads = threads;
        this.avgTime = avgTime;
        this.minTime = minTime;
        this.maxTime = maxTime;
    }
}
}

```

```

=== ./main/java/ua/kpi/lab8/controller/MatrixController.java ===
package ua.kpi.lab8.controller;

```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ua.kpi.lab8.model.ClientMultiplyRequest;
import ua.kpi.lab8.model.MultiplyResponse;
import ua.kpi.lab8.model.ServerMultiplyRequest;
import ua.kpi.lab8.service.MatrixService;

```

```

@RestController
@RequestMapping("/api/multiply")
public class MatrixController {

    private final MatrixService matrixService;

    public MatrixController(MatrixService matrixService) {
        this.matrixService = matrixService;
    }

    @PostMapping("/server")
    public ResponseEntity<MultiplyResponse>
multiplyServerSide(@RequestBody ServerMultiplyRequest request) {
        try {

```

```

        MultiplyResponse response =
matrixService.multiplyServerSide(request.getN(), request.getQ());
        return ResponseEntity.ok(response);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().build();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
    }
}

@PostMapping("/client")
public ResponseEntity<MultiplyResponse>
multiplyClientSide(@RequestBody ClientMultiplyRequest request) {
    try {
        MultiplyResponse response = matrixService.multiplyClientSide(
            request.getMatrixA(),
            request.getMatrixB(),
            request.getQ()
        );
        return ResponseEntity.ok(response);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().build();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
    }
}

@GetMapping("/health")
public ResponseEntity<String> health() {
    return ResponseEntity.ok("Matrix Multiplication Server is
running");
}
}

```

```

=== ./main/java/ua/kpi/lab8/model/ClientMultiplyRequest.java ===
package ua.kpi.lab8.model;

```

```

public class ClientMultiplyRequest {
    private double[][] matrixA;
    private double[][] matrixB;
    private int q;

    public ClientMultiplyRequest() {
    }

    public ClientMultiplyRequest(double[][] matrixA, double[][] matrixB,
int q) {
        this.matrixA = matrixA;
        this.matrixB = matrixB;
        this.q = q;
    }

    public double[][] getMatrixA() {
        return matrixA;
    }
}

```

```

    }

    public void setMatrixA(double[][] matrixA) {
        this.matrixA = matrixA;
    }

    public double[][] getMatrixB() {
        return matrixB;
    }

    public void setMatrixB(double[][] matrixB) {
        this.matrixB = matrixB;
    }

    public int getQ() {
        return q;
    }

    public void setQ(int q) {
        this.q = q;
    }
}

=== ./main/java/ua/kpi/lab8/model/MultiplyResponse.java ===
package ua.kpi.lab8.model;

public class MultiplyResponse {
    private double[][] result;
    private long computationTimeMs;
    private int matrixSize;
    private int gridSize;
    private int threadsUsed;
    private String dataSource;

    public MultiplyResponse() {
    }

    public MultiplyResponse(double[][] result, long computationTimeMs, int
matrixSize, int gridSize, int threadsUsed, String dataSource) {
        this.result = result;
        this.computationTimeMs = computationTimeMs;
        this.matrixSize = matrixSize;
        this.gridSize = gridSize;
        this.threadsUsed = threadsUsed;
        this.dataSource = dataSource;
    }

    public double[][] getResult() {
        return result;
    }

    public void setResult(double[][] result) {
        this.result = result;
    }

    public long getComputationTimeMs() {
        return computationTimeMs;
    }
}

```

```

    public void setComputationTimeMs(long computationTimeMs) {
        this.computationTimeMs = computationTimeMs;
    }

    public int getMatrixSize() {
        return matrixSize;
    }

    public void setMatrixSize(int matrixSize) {
        this.matrixSize = matrixSize;
    }

    public int getGridSize() {
        return gridSize;
    }

    public void setGridSize(int gridSize) {
        this.gridSize = gridSize;
    }

    public int getThreadsUsed() {
        return threadsUsed;
    }

    public void setThreadsUsed(int threadsUsed) {
        this.threadsUsed = threadsUsed;
    }

    public String getDataSource() {
        return dataSource;
    }

    public void setDataSource(String dataSource) {
        this.dataSource = dataSource;
    }
}

```

```

=== ./main/java/ua/kpi/lab8/model/ServerMultiplyRequest.java ===
package ua.kpi.lab8.model;

```

```

public class ServerMultiplyRequest {
    private int n;
    private int q;

    public ServerMultiplyRequest() {
    }

    public ServerMultiplyRequest(int n, int q) {
        this.n = n;
        this.q = q;
    }

    public int getN() {
        return n;
    }

    public void setN(int n) {

```



```

        this.n = n;
    }

    public int getQ() {
        return q;
    }

    public void setQ(int q) {
        this.q = q;
    }
}

=== ./main/java/ua/kpi/lab8/service/MatrixService.java ===
package ua.kpi.lab8.service;

import org.springframework.stereotype.Service;
import ua.kpi.lab8.algorithm.FoxParallelMultiplier;
import ua.kpi.lab8.model.MultiplyResponse;

import java.util.Random;

@Service
public class MatrixService {

    private final Random random = new Random();

    public MultiplyResponse multiplyServerSide(int n, int q) throws
    InterruptedException {
        double[][] matrixA = generateRandomMatrix(n);
        double[][] matrixB = generateRandomMatrix(n);

        return multiplyMatrices(matrixA, matrixB, q, "server");
    }

    public MultiplyResponse multiplyClientSide(double[][] matrixA,
    double[][] matrixB, int q) throws InterruptedException {
        validateMatrices(matrixA, matrixB);
        return multiplyMatrices(matrixA, matrixB, q, "client");
    }

    private MultiplyResponse multiplyMatrices(double[][] matrixA,
    double[][] matrixB, int q, String dataSource) throws InterruptedException
    {
        int n = matrixA.length;
        double[][] result = new double[n][n];

        FoxParallelMultiplier multiplier = new FoxParallelMultiplier(q);

        long startTime = System.currentTimeMillis();
        multiplier.multiply(matrixA, matrixB, result);
        long endTime = System.currentTimeMillis();

        long computationTime = endTime - startTime;
        int threadsUsed = multiplier.getWorkerCount();

        logStatistics(n, q, threadsUsed, computationTime, dataSource);
    }

```

```

        return new MultiplyResponse(result, computationTime, n, q,
threadsUsed, dataSource);
    }

    private void logStatistics(int n, int q, int threads, long timeMs,
String dataSource) {
        long operations = (long) n * n * n;
        double opsPerSecond = (operations / (timeMs / 1000.0));
        String opsFormatted = formatOps(opsPerSecond);

        System.out.printf("[BENCHMARK] %s-side | n=%d | q=%d | threads=%d
| time=%dms | ops=%s%n",
            capitalizeFirst(dataSource), n, q, threads, timeMs,
opsFormatted);
    }

    private String formatOps(double opsPerSecond) {
        if (opsPerSecond >= 1_000_000_000) {
            return String.format("%.2fG/s", opsPerSecond / 1_000_000_000);
        } else if (opsPerSecond >= 1_000_000) {
            return String.format("%.2fM/s", opsPerSecond / 1_000_000);
        } else if (opsPerSecond >= 1_000) {
            return String.format("%.2fK/s", opsPerSecond / 1_000);
        } else {
            return String.format("%.2f/s", opsPerSecond);
        }
    }

    private String capitalizeFirst(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }
        return str.substring(0, 1).toUpperCase() + str.substring(1);
    }

    private double[][] generateRandomMatrix(int n) {
        double[][] matrix = new double[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = random.nextDouble() * 10;
            }
        }
        return matrix;
    }

    private void validateMatrices(double[][] matrixA, double[][] matrixB)
{
        if (matrixA == null || matrixB == null) {
            throw new IllegalArgumentException("Matrices cannot be null");
        }
        if (matrixA.length == 0 || matrixB.length == 0) {
            throw new IllegalArgumentException("Matrices cannot be
empty");
        }
        if (matrixA.length != matrixA[0].length || matrixB.length !=
matrixB[0].length) {
            throw new IllegalArgumentException("Matrices must be square");
        }
    }

```

```
        if (matrixA.length != matrixB.length) {
            throw new IllegalArgumentException("Matrices must have the
same dimensions");
        }
    }
}
```